

Resumen Teórico: Indecidibilidad y Diagonalización

Computabilidad y Complejidad — UNRC

Basado en el material de Pablo Castro (UNRC)

Índice

1. Introducción	1
2. Problemas decidibles vs. indecidibles	1
2.1. Codificar problemas como lenguajes	1
3. Algunos problemas decidibles	2
3.1. Vacío del lenguaje de un AFD	2
3.2. Pertenencia para gramáticas libres de contexto	2
4. Problemas indecidibles: primer encuentro	3
4.1. A_{TM} es Turing reconocible	3
5. Cardinalidad de conjuntos infinitos	4
5.1. Igualdad y desigualdad de cardinales	4
5.2. Aritmética transfinita	4
5.3. Conjuntos contables	4
6. La diagonalización de Cantor: $\mathbb{N} < \mathbb{R}$	5
6.1. Reducción al intervalo $[0, 1)$	5
6.2. La prueba diagonal	5
7. Existencia de lenguajes no computables	6
8. El Halting Problem: A_{TM} es indecidible	6
8.1. Enunciado y suposición por absurdo	7
8.2. Construcción de la máquina diagonal D	7
8.3. Contradicción: ejecutar D sobre $\langle D \rangle$	7
8.4. ¿Por qué se llama “diagonalización”?	7
9. Un lenguaje que ni siquiera es reconocible	8
9.1. Un teorema clave	8
9.2. $\overline{A_{TM}}$ no es Turing reconocible	8
10. Resumen final	9

1. Introducción

En el material anterior vimos cómo las máquinas de Turing capturan formalmente la noción de cómputo. Llegó la hora de usar esa formalización para demostrar el resultado más sorprendente

de la teoría de la computabilidad: **existen problemas que ninguna máquina de Turing puede resolver.**

Este resumen recorre el siguiente camino:

1. **Cómo codificar problemas como lenguajes:** para hablar formalmente de “problemas computables”.
2. **Ejemplos de problemas decidibles:** el vacío de un AFD, la pertenencia para gramáticas libres de contexto.
3. **Cardinalidad de conjuntos infinitos** (Cantor): para mostrar que existen *muchos* lenguajes no computables.
4. **La técnica de diagonalización:** usada por Cantor para probar que $\mathbb{R}$ no es contable, y por Turing para mostrar que el *Halting Problem* es indecidible.
5. **Un lenguaje que ni siquiera es reconocible:** hay problemas más difíciles que el Halting Problem.

2. Problemas decidibles vs. indecidibles

2.1. Codificar problemas como lenguajes

Para hablar formalmente de “problemas computables”, representamos cada *problema* como un *lenguaje*: el conjunto de entradas codificadas que constituyen “instancias positivas” del problema. Resolver el problema equivale entonces a decidir el lenguaje asociado.

Ejemplo 2.1 (Aceptación de una MT). El problema “¿la máquina M acepta la palabra w ?” se codifica como

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ es una MT y } M \text{ acepta } w\}.$$

Si pudiéramos decidir este lenguaje, tendríamos un algoritmo general para predecir si cualquier MT acepta o no una cadena dada.

Ejemplo 2.2 (Satisfacibilidad). El problema SAT se codifica como

$$SAT = \{\langle \phi \rangle \mid \phi \text{ es una fórmula proposicional satisfacible}\}.$$

Decidir este lenguaje sería tener un programa que dice si una fórmula es satisfacible.

Intuición

La idea es siempre la misma: tomamos un problema (“dado un input, responder Sí/No”) y lo convertimos en un lenguaje (“el conjunto de inputs cuya respuesta es Sí”). De este modo todas las preguntas sobre algoritmos se vuelven preguntas sobre lenguajes y máquinas de Turing.

3. Algunos problemas decidibles

Antes de meternos con los problemas indecidibles, conviene ver ejemplos no triviales de problemas *decidibles*. Esto sirve también para fijar la técnica de “construir una MT que decide un lenguaje”.

3.1. Vacío del lenguaje de un AFD

Teorema 3.1. El lenguaje

$$E_{DFA} = \{\langle A \rangle \mid A \text{ es un AFD y } L(A) = \emptyset\}$$

es decidable.

Idea. Un AFD acepta alguna cadena si y solo si *algún estado final es alcanzable* desde el estado inicial. Por lo tanto, basta con un algoritmo de alcanzabilidad sobre el grafo del autómata.

Algoritmo (máquina M que decide E_{DFA}). Dada la representación $\langle A \rangle$ de un AFD:

1. Marcar el estado inicial de A .
2. Repetir mientras se agregan nuevos estados marcados:
 - Marcar todo estado que esté conectado (por alguna transición) a un estado ya marcado.
3. Cuando ya no se agregan nuevos estados marcados:
 - Si *ningún* estado final quedó marcado, aceptar (el lenguaje es vacío).
 - Si algún estado final fue marcado, rechazar.

El algoritmo termina porque la cantidad de estados es finita y a lo sumo se hacen tantas iteraciones como estados tenga A . Es, por lo tanto, una máquina de decisión.

3.2. Pertenencia para gramáticas libres de contexto

Teorema 3.2. El lenguaje

$$A_{CFG} = \{\langle G, w \rangle \mid G \text{ es una CFG y } G \text{ genera } w\}$$

es decidable.

Idea. Si pudiéramos enumerar *todas* las derivaciones de G y verificar si alguna produce w , decidiríamos el problema. El obstáculo es que una CFG arbitraria tiene infinitas derivaciones posibles. La clave es usar la **Forma Normal de Chomsky**, en la que toda derivación de una palabra de tamaño n tiene una cantidad acotada de pasos.

Definición 3.3 (Forma Normal de Chomsky). Una CFG está en **forma normal de Chomsky** si todas sus producciones tienen alguna de las formas

$$A \rightarrow BC \quad \text{o} \quad A \rightarrow a,$$

donde A, B, C son variables y a es un terminal.

Proposición 3.4 (Cota sobre las derivaciones). Si G está en forma normal de Chomsky y w tiene longitud n , entonces toda derivación de w en G tiene exactamente $2n - 1$ pasos.

Por qué es cierto. Cada producción $A \rightarrow BC$ aumenta en uno la cantidad de variables; cada producción $A \rightarrow a$ “convierte” una variable en un terminal. Para llegar a n terminales hace falta primero generar n variables (a partir de una inicial, en $n - 1$ aplicaciones de reglas tipo BC) y luego convertir cada una en un terminal (n aplicaciones del segundo tipo). Total: $(n - 1) + n = 2n - 1$.

Algoritmo (máquina M que decide A_{CFG}). Dada la entrada $\langle G, w \rangle$ con $|w| = n$:

1. Convertir G a forma normal de Chomsky.
2. Generar todas las derivaciones de exactamente $2n - 1$ pasos (mediante un DFS acotado sobre el árbol de derivaciones).
3. Si alguna de ellas produce w , aceptar; si no, rechazar.

El paso (2) termina porque el árbol de derivaciones de profundidad $2n - 1$ es finito.

Intuición

La estrategia general en estas pruebas es la misma: encontrar una *cota* sobre el espacio de búsqueda. Cuando hay una cota finita y computable, el problema es decidible porque podemos “probar todas las opciones”. La indecidibilidad va a aparecer justamente cuando *no* podamos acotar el espacio de búsqueda.

4. Problemas indecidibles: primer encuentro

Hay problemas sumamente naturales que, sin embargo, son *indecidibles*. El ejemplo paradigmático es:

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ es una MT y } M \text{ acepta } w \}.$$

4.1. A_{TM} es Turing reconocible

Aunque no sea decidible, A_{TM} *sí* es Turing reconocible. La idea es usar una **máquina universal** U que simula a cualquier otra MT.

Construcción de U . Dada la entrada $\langle M, w \rangle$:

1. U simula paso a paso la ejecución de M con entrada w .
2. Si en algún momento M acepta, U acepta.
3. Si M rechaza, U rechaza.

Observación 4.1. Si M no termina con la entrada w , entonces U tampoco termina al simularlo. Esto significa que U *reconoce* A_{TM} (acepta exactamente las cadenas $\langle M, w \rangle$ con $w \in \mathcal{L}(M)$) pero no es una máquina de decisión: no podemos garantizar terminación cuando $w \notin \mathcal{L}(M)$.

Intuición

La existencia de U es una idea muy poderosa: prueba que “ejecutar programas” es algo que puede hacer una MT. En cierto sentido, U es el ancestro teórico de las computadoras modernas, que ejecutan programas arbitrarios codificados como datos.

5. Cardinalidad de conjuntos infinitos

Antes de demostrar la indecidibilidad de A_{TM} , conviene desarrollar algunas herramientas de Cantor para comparar conjuntos infinitos. Estas ideas son el corazón del argumento diagonal.

5.1. Igualdad y desigualdad de cardinales

Definición 5.1 (Misma cardinalidad). Dos conjuntos A y B tienen la misma cardinalidad, escrito $A \cong B$, si existe una función **biyectiva** $f : A \rightarrow B$.

Definición 5.2 (Estrictamente menor). $A < B$ (cardinalmente) si existe una función inyectiva $A \rightarrow B$, pero *no* existe ninguna biyección.

Intuición

Para conjuntos finitos esto coincide con la idea usual de “ A tiene menos elementos que B ”. Para conjuntos infinitos puede dar resultados sorprendentes: por ejemplo, $\mathbb{N}$ tiene la misma cardinalidad que $\mathbb{Z}$ y que $\mathbb{Q}$, pero *menor* cardinalidad que $\mathbb{R}$.

5.2. Aritmética transfinita

La cardinalidad de un conjunto A se nota $\#A$. Las cardinalidades de los conjuntos infinitos se denotan con la letra hebrea $\aleph$ (*aleph*):

$$\aleph_0 = \#\mathbb{N}, \quad \aleph_1 = \text{“el siguiente infinito”}, \quad \aleph_2, \aleph_3, \dots$$

Teorema 5.3 (Cantor). Para todo conjunto A ,

$$A < \mathcal{P}(A),$$

es decir, el conjunto de partes siempre tiene *estrictamente* más elementos que el conjunto original.

Existen, por lo tanto, infinitos cardinales infinitos. La relación exacta entre $\aleph_1$ y $\mathcal{P}(\aleph_0)$ depende de la axiomatización de la teoría de conjuntos que se use (es el famoso *problema del continuo*).

5.3. Conjuntos contables

Definición 5.4 (Contable). Un conjunto A es **contable** si tiene la misma cardinalidad que $\mathbb{N}$, es decir, si existe una biyección entre A y $\mathbb{N}$.

Ejemplo 5.5 ($\mathbb{N} \cong \mathbb{Z}$). La función $f : \mathbb{N} \rightarrow \mathbb{Z}$ definida por

$$f(n) = \begin{cases} \lceil n/2 \rceil & \text{si } n \text{ es impar,} \\ -n/2 & \text{si } n \text{ es par,} \end{cases}$$

es biyectiva. Listada:

$$0 \mapsto 0, \quad 1 \mapsto 1, \quad 2 \mapsto -1, \quad 3 \mapsto 2, \quad 4 \mapsto -2, \quad 5 \mapsto 3, \dots$$

Por lo tanto $\mathbb{Z}$ es contable.

Ejemplo 5.6 ($\mathbb{N} \cong \mathbb{Q}$). Los racionales positivos se pueden poner en una grilla infinita

$$\begin{array}{ccccccc} 1/1 & 1/2 & 1/3 & 1/4 & \dots & & \\ 2/1 & 2/2 & 2/3 & 2/4 & \dots & & \\ 3/1 & 3/2 & 3/3 & 3/4 & \dots & & \\ 4/1 & 4/2 & 4/3 & 4/4 & \dots & & \\ \vdots & \vdots & \vdots & \vdots & & & \end{array}$$

y enumerarse recorriendo las diagonales $(1/1, 1/2, 2/1, 1/3, 2/2, 3/1, \dots)$. Como cada racional aparece en la grilla, podemos definir una sobreyección $\mathbb{N} \rightarrow \mathbb{Q}^+$. Saltando duplicados y combinando con los negativos se obtiene una biyección $\mathbb{N} \cong \mathbb{Q}$.

Idea clave

Para mostrar que un conjunto es contable, basta con *enumerar* sus elementos: dar una forma de listarlos $a_0, a_1, a_2, \dots$ de modo que cada elemento aparezca al menos una vez.

6. La diagonalización de Cantor: $\mathbb{N} < \mathbb{R}$

Ahora viene el argumento más importante del PDF: existe un conjunto, $\mathbb{R}$, que es *estrictamente* mayor que $\mathbb{N}$. La técnica que se usa para demostrarlo se llama **diagonalización**, y se va a reutilizar para probar la indecidibilidad del Halting Problem.

6.1. Reducción al intervalo $[0, 1)$

Basta con mostrar que el intervalo $[0, 1)$ no es contable, ya que si lo fuera, $\mathbb{R}$ tampoco podría serlo (porque $[0, 1) \subseteq \mathbb{R}$).

6.2. La prueba diagonal

Teorema 6.1 (Cantor). El intervalo $[0, 1)$ no es contable.

Prueba (por contradicción). Supongamos que $[0, 1)$ es contable. Entonces existe una biyección que enumera todos sus elementos como $r_1, r_2, r_3, \dots$. Escribiendo cada r_i en su representación decimal,

$$r_i = 0.d_i^1 d_i^2 d_i^3 d_i^4 \dots$$

podemos disponer todas las representaciones en una tabla (la fila i contiene los dígitos de r_i):

	1	2	3	4	...
r_1	d_1^1	d_1^2	d_1^3	d_1^4	...
r_2	d_2^1	d_2^2	d_2^3	d_2^4	...
r_3	d_3^1	d_3^2	d_3^3	d_3^4	...
r_4	d_4^1	d_4^2	d_4^3	d_4^4	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	

Construimos ahora un número $\bar{r} \in [0, 1)$, dígito a dígito, que *no* aparece en la lista. Definimos sus dígitos $\bar{d}^1, \bar{d}^2, \bar{d}^3, \dots$ con la regla

$$\bar{d}^i = \begin{cases} 1 & \text{si } d_i^i \neq 1, \\ 0 & \text{si } d_i^i = 1. \end{cases}$$

Observación clave. Por construcción, $\bar{d}^i \neq d_i^i$ para todo i . Por lo tanto $\bar{r}$ difiere de r_i *al menos* en el i -ésimo dígito decimal. Esto implica $\bar{r} \neq r_i$ para todo i .

Contradicción. Asumimos que la enumeración cubría *todo* $[0, 1)$, pero acabamos de construir un elemento de $[0, 1)$ que no aparece. Conclusión: no existe ninguna enumeración de $[0, 1)$, y por lo tanto $[0, 1)$ — y $\mathbb{R}$ — no son contables. $\square$

Intuición

La idea de la diagonalización es asumir que tenemos una enumeración *completa* y construir un elemento que difiera, en la posición i , del i -ésimo elemento enumerado. Cualquier intento de “haberlo incluido” falla, porque por la propia construcción nuestro elemento es distinto de cada uno de los enumerados.

7. Existencia de lenguajes no computables

Ahora aplicamos los resultados de cardinalidad a las máquinas de Turing.

Teorema 7.1. Existen lenguajes que no son Turing reconocibles.

Prueba (argumento de conteo).

- El conjunto $\{\langle T \rangle \mid T \text{ es una MT}\}$ es contable: $\aleph_0$. Cada MT se puede codificar como una cadena finita sobre un alfabeto fijo, y las cadenas finitas son contables.
- El conjunto $\{0, 1\}^*$ de *todas* las cadenas finitas también es contable: $\aleph_0$.
- Un *lenguaje* sobre $\{0, 1\}$ es un subconjunto de $\{0, 1\}^*$. El conjunto de todos los lenguajes es entonces $\mathcal{P}(\{0, 1\}^*)$.
- Por el teorema de Cantor: $\#\{0, 1\}^* < \#\mathcal{P}(\{0, 1\}^*)$.

Resumiendo: $\#\{\langle M \rangle \mid M \text{ es MT}\} = \aleph_0 < \#\mathcal{P}(\{0, 1\}^*)$. Hay *estrictamente más lenguajes que máquinas de Turing*. Por lo tanto, debe haber lenguajes que ninguna MT reconoce. $\square$

Idea clave

Este argumento prueba la *existencia* de lenguajes no computables sin exhibir uno concreto. Es un resultado puramente de cardinalidad: hay tantos lenguajes que las MTs no alcanzan a cubrirlos todos. En las próximas secciones vamos a *exhibir* ejemplos concretos.

8. El Halting Problem: A_{TM} es indecidible

Ahora viene el resultado más famoso de la materia. Vamos a mostrar que el lenguaje $A_{TM} = \{\langle M, w \rangle \mid M \text{ acepta } w\}$ es indecidible.

8.1. Enunciado y suposición por absurdo

Teorema 8.1 (Turing). El lenguaje A_{TM} es indecidible.

Prueba (por contradicción). Supongamos que A_{TM} es decidable. Entonces existe una máquina de Turing H que *siempre termina* y satisface

$$H(\langle M, w \rangle) = \begin{cases} \text{acepta} & \text{si } M \text{ acepta } w, \\ \text{rechaza} & \text{si } M \text{ no acepta } w. \end{cases}$$

8.2. Construcción de la máquina diagonal D

Usando H como subrutina, construimos una nueva máquina D que recibe como entrada la codificación $\langle M \rangle$ de una MT y procede así:

1. Simula H con la entrada $\langle M, \langle M \rangle \rangle$ (es decir, H pregunta si M acepta su propia descripción).
2. Si H acepta, entonces D **rechaza**.
3. Si H rechaza, entonces D **acepta**.

En símbolos:

$$D(\langle M \rangle) = \begin{cases} \text{acepta} & \text{si } M \text{ no acepta } \langle M \rangle, \\ \text{rechaza} & \text{si } M \text{ acepta } \langle M \rangle. \end{cases}$$

8.3. Contradicción: ejecutar D sobre $\langle D \rangle$

Ahora preguntamos: ¿qué hace D con la entrada $\langle D \rangle$?

- Si D *acepta* $\langle D \rangle$, entonces por su definición D no acepta $\langle D \rangle$. Contradicción.
- Si D *rechaza* $\langle D \rangle$, entonces por su definición D acepta $\langle D \rangle$. Contradicción.

En cualquier caso llegamos a una contradicción. La hipótesis de partida (que existe H) debe ser falsa: no puede haber ninguna MT que decida A_{TM} . $\square$

8.4. ¿Por qué se llama “diagonalización”?

La conexión con el argumento de Cantor se ve claramente al imaginar una tabla infinita en la que la fila i corresponde a la máquina M_i y la columna j a la entrada $\langle M_j \rangle$, con un símbolo A o R en cada celda según M_i acepte o rechace $\langle M_j \rangle$:

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\dots$	$\langle D \rangle$
M_1	A	R	R	$\dots$	
M_2	R	R	A	$\dots$	
M_3	A	R	A	$\dots$	
M_4	R	R	R	R	$\dots$
$\vdots$					
D					?

La máquina H , si existiera, *completaría toda la grilla*: para cualquier par (i, j) podría decirnos si M_i acepta $\langle M_j \rangle$.

La máquina D está diseñada para *contradecir la diagonal*: D acepta $\langle M_i \rangle$ si y solo si M_i rechaza $\langle M_i \rangle$. Es decir, D es “lo opuesto a la diagonal”.

Pero D es ella misma una MT, así que aparece como una fila en la tabla, digamos $M_k = D$. ¿Qué tendría que valer en la celda $(D, \langle D \rangle)$? Por construcción, D acepta $\langle D \rangle$ si y solo si D rechaza $\langle D \rangle$. Contradicción.

Intuición

La construcción es exactamente análoga a la de Cantor: en la celda (i, i) ponemos lo *opuesto* a lo que dice la fila i , y obtenemos un elemento que no puede estar en la lista. La existencia de H permitiría construir D , y D es la “anti-diagonal” que rompe todo.

9. Un lenguaje que ni siquiera es reconocible

El Halting Problem mostró que existen lenguajes Turing reconocibles que no son decidibles. ¿Habrán lenguajes que ni siquiera sean reconocibles? Sí.

9.1. Un teorema clave

Teorema 9.1. Si un lenguaje L es Turing reconocible y su complemento $\bar{L}$ también lo es, entonces L es decidable.

Prueba. Supongamos que M reconoce L y que $\bar{M}$ reconoce $\bar{L}$. Construimos una máquina T con dos cintas auxiliares que decide L :

- En la cinta 2 simula M con la entrada original.
- En la cinta 3 simula $\bar{M}$ con la misma entrada.
- Las simulaciones avanzan *en paralelo*, alternando un paso de M y un paso de $\bar{M}$.
- Si en algún momento la simulación de M acepta, T acepta.
- Si en algún momento la simulación de $\bar{M}$ acepta, T rechaza.

¿Por qué T siempre termina? Dada una entrada w , hay dos casos:

- Si $w \in L$, entonces M acepta w en una cantidad finita de pasos, y T termina aceptando.
- Si $w \notin L$, entonces $w \in \overline{L}$, y por hipótesis $\overline{M}$ acepta w en una cantidad finita de pasos, y T termina rechazando.

En cualquier caso T termina con la respuesta correcta. Por lo tanto L es decidible. $\square$

9.2. $\overline{A_{TM}}$ no es Turing reconocible

Definamos el complemento del Halting Problem:

$$\overline{A_{TM}} = \{\langle M, w \rangle \mid M \text{ no acepta } w\}.$$

Teorema 9.2. $\overline{A_{TM}}$ no es Turing reconocible.

Prueba (por contradicción). Supongamos que $\overline{A_{TM}}$ es Turing reconocible. Ya vimos que A_{TM} también es Turing reconocible (vía la máquina universal U). Entonces, por el Teorema 9.1, A_{TM} sería decidible. Pero acabamos de demostrar que A_{TM} no es decidible. Contradicción. $\square$

Idea clave

Tenemos entonces una jerarquía estricta:

$$\text{decidibles} \subsetneq \text{Turing reconocibles} \subsetneq \text{todos los lenguajes}.$$

- Los lenguajes **decidibles** son los que algún algoritmo siempre puede responder.
- Los **Turing reconocibles** son los que algún algoritmo puede *confirmar* cuando la respuesta es Sí, pero pueden no terminar si la respuesta es No.
- Los lenguajes que **no son ni siquiera reconocibles** son aquellos para los que ni siquiera podemos pretender confirmar las respuestas afirmativas. $\overline{A_{TM}}$ es el primer ejemplo concreto.

10. Resumen final

Las ideas y resultados más importantes para recordar:

1. **Codificación de problemas como lenguajes.** Decidir si un problema es algorítmicamente resoluble equivale a decidir si su lenguaje asociado es *decidible*.
2. **Problemas decidibles concretos.** Tanto el vacío de un AFD (E_{DFA}) como la pertenencia para gramáticas libres de contexto (A_{CFG}) son decidibles. La técnica común es *acotar el espacio de búsqueda* (BFS sobre estados, derivaciones de longitud $2n - 1$ en forma normal de Chomsky).
3. **La máquina universal U .** Dada $\langle M, w \rangle$, U simula M con entrada w . Esto muestra que A_{TM} es Turing reconocible (aunque no decidible).
4. **Cardinalidad de Cantor.** Dos conjuntos tienen la misma cardinalidad si hay una biyección entre ellos. $\mathbb{N} \cong \mathbb{Z} \cong \mathbb{Q}$ son todos contables; en cambio, $\mathbb{R}$ es *estrictamente* más grande ($\mathbb{N} < \mathbb{R}$).
5. **Argumento diagonal de Cantor.** La no-contabilidad de $\mathbb{R}$ se prueba suponiendo una enumeración y construyendo un nuevo número que difiere del i -ésimo en su i -ésimo dígito.

6. **Existencia de lenguajes no computables.** Hay $\aleph_0$ máquinas de Turing y $2^{\aleph_0}$ lenguajes posibles. Por simple conteo, deben existir lenguajes no reconocibles.
7. **Indecidibilidad de A_{TM} (Halting Problem).** Si existiera un decididor H , podríamos construir una máquina D que acepta $\langle M \rangle$ si y solo si M no acepta $\langle M \rangle$. Evaluar $D(\langle D \rangle)$ produce contradicción.
8. **Teorema del complemento.** Si L y $\bar{L}$ son ambos reconocibles, entonces L es decidable. Equivalentemente: si L es reconocible pero no decidable, entonces $\bar{L}$ *no* es reconocible.
9. **$\overline{A_{TM}}$ no es reconocible.** Es el primer ejemplo concreto de lenguaje fuera de la clase Turing reconocible.
10. **Jerarquía estricta.** Los lenguajes decidibles son un subconjunto propio de los reconocibles, que a su vez son un subconjunto propio de todos los lenguajes.

Intuición

La gran lección de este teórico es que la noción de cómputo, formalizada por las MTs, tiene *límites intrínsecos*: no porque seamos malos programadores ni porque nos falte memoria, sino por razones puramente matemáticas. La diagonalización es la herramienta universal para exhibir esos límites: aparece en Cantor, en Turing, en Gödel, y en muchos otros lugares de la lógica y la computabilidad.